Lessons from designing better tabular neural networks

AutoML School Tübingen 2025

David Holzmüller (INRIA Paris/Saclay)

June 10, 2025

What is tabular data?

Age	Gender	Weight	Time in Hospital	...	Readmitted?
60	Male	90	5 days	...	Yes
45	Female	N/A	3 days	...	No
⋮	⋮	⋮	⋮		⋮

Also called *structured data*

- Fixed number of features/columns
- Heterogeneous column types: Numerical, categorical, bool, text, date, timespan
- No spatiotemporal structure that could be exploited by more specialized methods
- Missing values

Clasification and regression on tabular data: why?

- Probably the most common form of data in practical ML
  - $\rightsquigarrow$ healthcare, science (surrogate models), retail, energy, banking, finance, ...
- Benefits from AutoML
  - Relatively cheap methods
  - Diversity of methods
  - Diversity of datasets
- Tabular data for AutoML: surrogate models for hyperparameter optimization (HPO)

How to represent tabular data in python?

Using `pandas` (popular) or `polars` (fast)!

```python
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'num_1': [0.1, 0.2, np.NaN],
    'cat_1': ['blue', np.NaN, 'green'],
    'y': [1.0, 2.0, 3.0]
})
print(df)
#    num_1  cat_1    y
# 0    0.1   blue  1.0
# 1    0.2    NaN  2.0
# 2    NaN  green  3.0
```

How to represent tabular data in python?

Using `pandas` (popular) or `polars` (fast)!

```python
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'num_1': [0.1, 0.2, np.NaN],
    'cat_1': ['blue', np.NaN, 'green'],
    'y': [1.0, 2.0, 3.0]
})
print(df)
#    num_1  cat_1    y
# 0    0.1   blue  1.0
# 1    0.2    NaN  2.0
# 2    NaN  green  3.0
```

Tip

Use dataframes to store/process

- (original) datasets
- benchmark results

State of neural networks in 2022

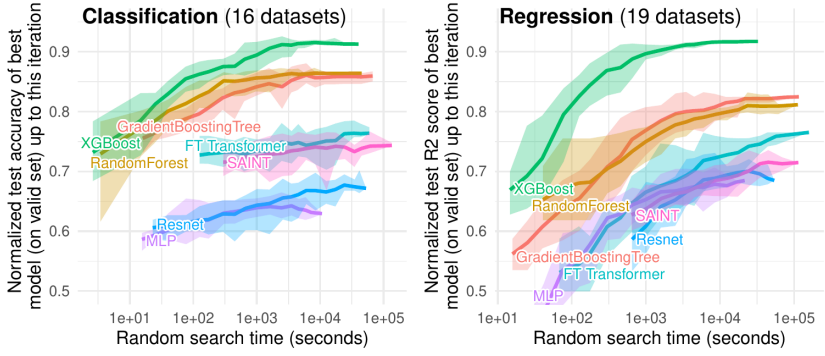


Figure 6: **Time benchmark on medium-sized datasets, with only numerical features**. The first random search iteration corresponds to default hyperparameters. Each value corresponds to the test score of the best model (on the validation set) after a specific time spent doing random search, averaged on 15 shuffles of the random search order. The ribbon corresponds to the minimum and maximum scores on these 15 shuffles.

Source: Grinsztajn et al. (2022)

Gradient-boosted decision trees

- Sequentially learn an ensemble of decision trees by optimizing a Taylor approximation of the loss
- Popular: XGBoost, LightGBM, CatBoost, sklearn HistGradientBoosting

Why do they work so well (Grinsztajn et al., 2022)?

- Ability to ignore irrelevant features
- Learn axis-aligned functions
- Ability to learn non-smooth / high-frequency functions
- Ensembles deal well with noisy labels

Why neural networks?

- Flexibility (architecture / coding in python)
- Multimodal data
- Leveraging advances in NN research (pre-training, semi-supervised learning etc.)
- Smooth solutions, differentiability
- Extrapolation
- Complement boosted trees well

Why neural networks?

- Flexibility (architecture / coding in python)
- Multimodal data
- Leveraging advances in NN research (pre-training, semi-supervised learning etc.)
- Smooth solutions, differentiability
- Extrapolation
- Complement boosted trees well

Problem

NNs are relatively slow compared to boosted trees $\rightsquigarrow$ can be annoying with hyperparameter tuning

Why neural networks?

- Flexibility (architecture / coding in python)
- Multimodal data
- Leveraging advances in NN research (pre-training, semi-supervised learning etc.)
- Smooth solutions, differentiability
- Extrapolation
- Complement boosted trees well

Problem

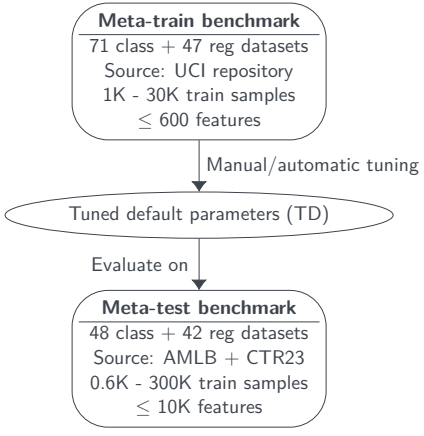
NNs are relatively slow compared to boosted trees $\rightsquigarrow$ can be annoying with hyperparameter tuning

Paper: David Holzmüller, Léo Grinsztajn, Ingo Steinwart, Better by Default: Strong Pre-Tuned MLPs and Boosted Trees on Tabular Data, NeurIPS 2024.

Meta-learning

"Learning how to learn": each sample is a dataset

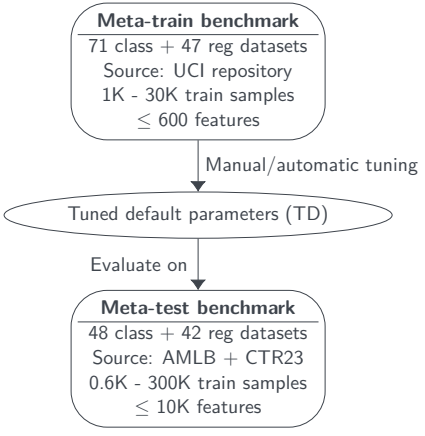
- here: fixed set of hyperparameters
- portfolio (list of models + hyperparameters)
- dataset properties $\mapsto$ hyperparameters
- TabPFN (Hollmann et al., 2025): $(\mathcal{D}_{\text{train}}, x_{\text{test}}) \mapsto \hat{y}_{\text{test}}$



Meta-learning

"Learning how to learn": each sample is a dataset

- here: fixed set of hyperparameters
- portfolio (list of models + hyperparameters)
- dataset properties $\mapsto$ hyperparameters
- TabPFN (Hollmann et al., 2025): $(\mathcal{D}_{\text{train}}, x_{\text{test}}) \mapsto \hat{y}_{\text{test}}$



Need a score to optimize

- Base metric: classification error / RMSE
- Aggregation: Geometric mean of (error + 0.01)
  - Values *relative* instead of absolute error reduction
  - Aggregation doesn't depend on other baselines $\rightarrow$ good to optimize

Meta-learning: Manual or automated?

- GBDTs: trying automated (random search, hyperopt, SMAC3)
- partially disappointing: 100 steps sometimes worse than ≈25 steps of manual exploration
- (wrote fancy coordinate descent optimizer but didn't study it much further)
- NNs: mostly manual (want to get hands-on experience, want to make time-related trade-offs)

Meta-learning: Manual or automated?

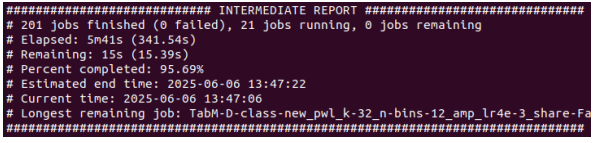
- GBDTs: trying automated (random search, hyperopt, SMAC3)
- partially disappointing: 100 steps sometimes worse than ≈25 steps of manual exploration
- (wrote fancy coordinate descent optimizer but didn't study it much further)
- NNs: mostly manual (want to get hands-on experience, want to make time-related trade-offs)

Would be nice to have a tool that can do efficient, time-aware meta-learning with interpretable insights

Optimizing benchmark runtime

- Setup: 4x3090 GPUs, 32-core CPU
- Boosted trees: 64 processes with 1 thread each
- Neural nets:
  - Parallelize across GPUs
  - Multiple processes per GPU
  - Vectorize multiple train/val/test splits within the same model (ca 4x speedup on GPU?)
    - Problems: doesn't work for separate HPO, increases code complexity, doesn't work for all optimizers
- Wrote my own scheduler because I'm on a machine without slurm

A custom scheduler



- Schedule within python using ray, slowest jobs first, respect RAM and usage limits, supports multi-node + multi-GPU, remaining time estimation
- Also well-suited for quick benchmarks
- Generically usable: `https://pytabkit.readthedocs.io/en/latest/bench/using_the_scheduler.html`
- Requires methods to estimate their RAM usage $\rightsquigarrow$ linear quantile regression tool to fit coefficients (works semi-well)
- Needs methods whose resource usage can be controlled within python (not AutoGluon)

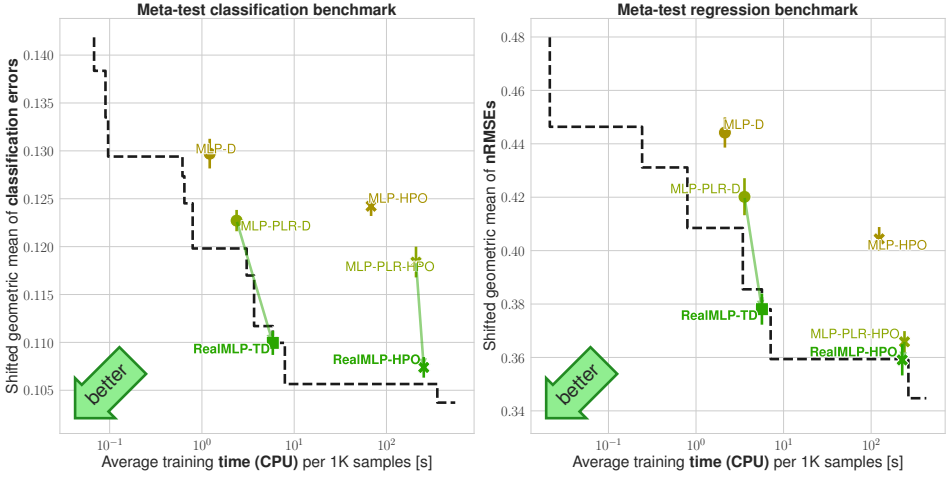
End result

71 datasets $\times$ 10 splits in $\approx$ 5 minutes (2 minutes for smaller MLPs)

HPO: Random search is just too convenient

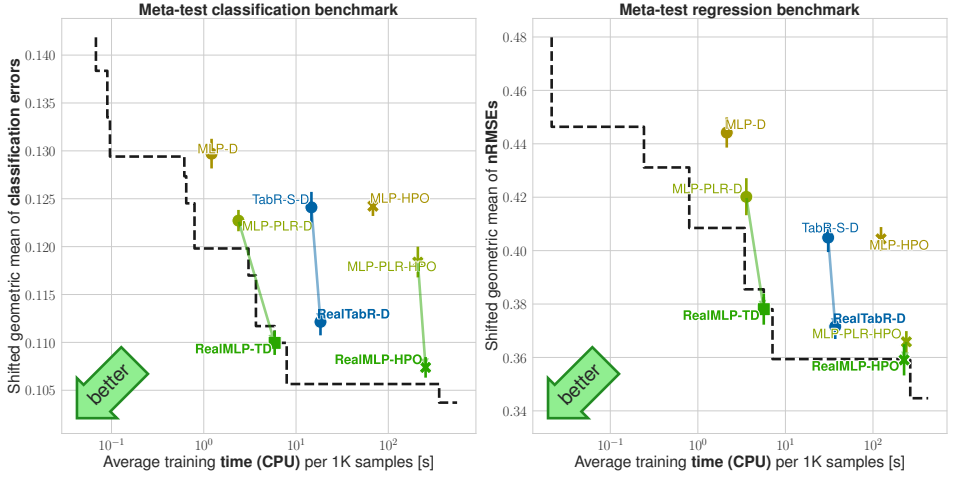
- fully parallelizable, no long sequential jobs
- works for multiple metrics simultaneously
- diverse configs for weighted ensembles
- easy to estimate runtime
- etc.

Results (Holzmüller et al., 2024)



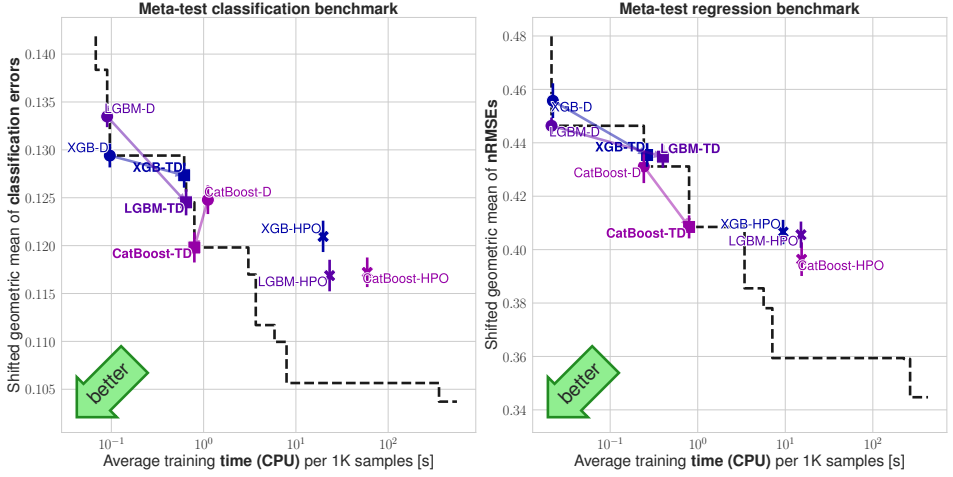
Get models + benchmark: `pip install pytabkit`

Results (Holzmüller et al., 2024)



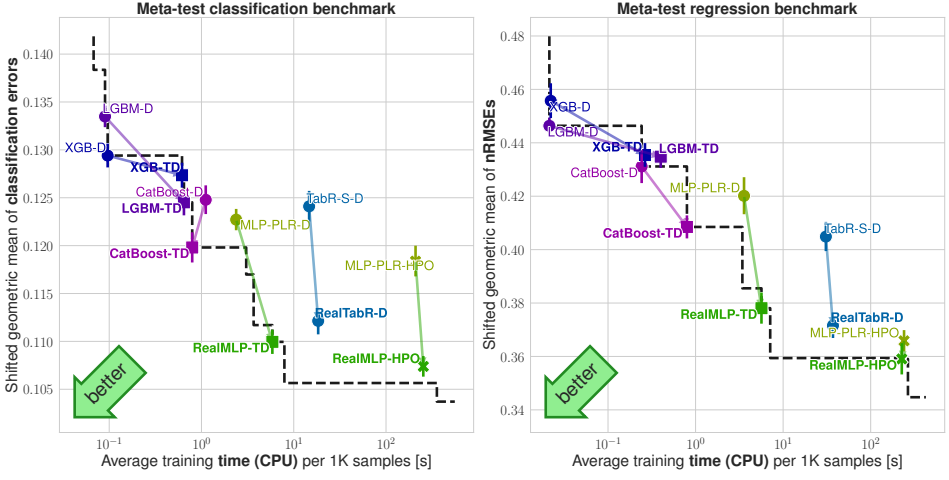
Get models + benchmark: `pip install pytabkit`

Results (Holzmüller et al., 2024)



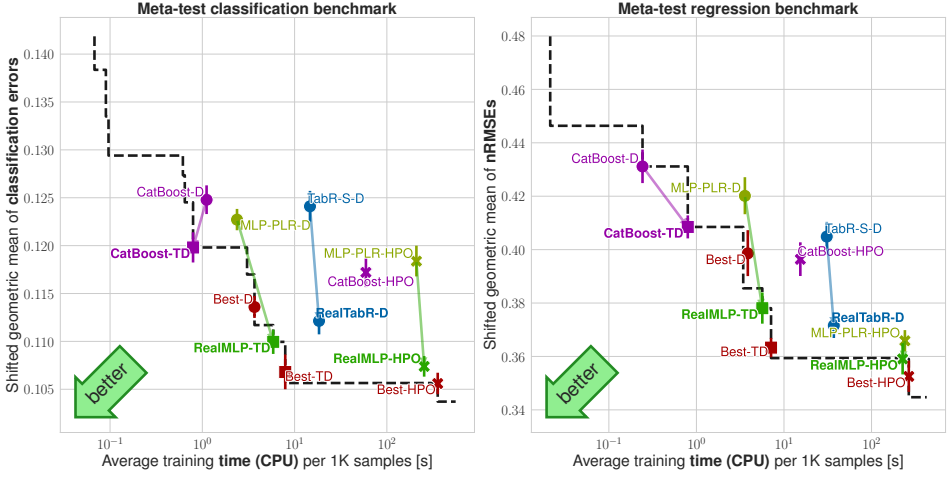
Get models + benchmark: `pip install pytabkit`

Results (Holzmüller et al., 2024)



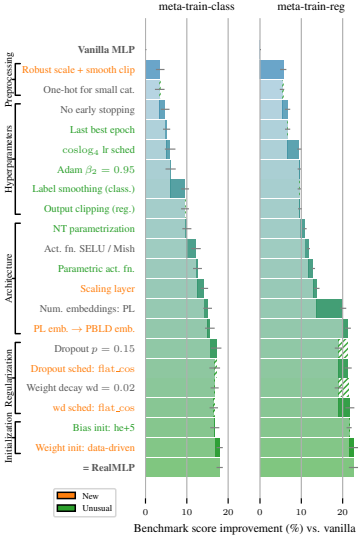
Get models + benchmark: `pip install pytabkit`

Results (Holzmüller et al., 2024)



Get models + benchmark: `pip install pytabkit`

RealMLP: A bag of tricks?



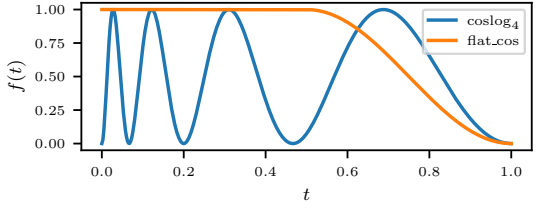
Difficult to get through reviews...

The early stopping trade-off

- Early stopping vs. best-epoch selection

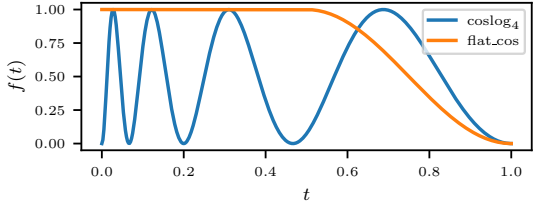
The early stopping trade-off

- Early stopping vs. best-epoch selection
- Benefits of (multi-cycle) schedules

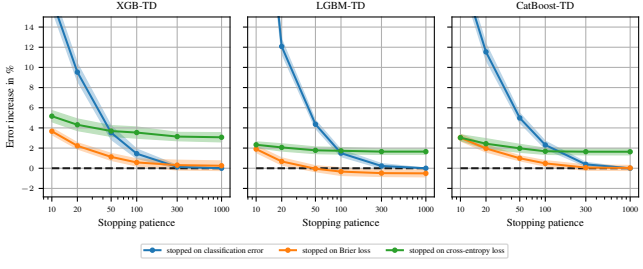


The early stopping trade-off

- Early stopping vs. best-epoch selection
- Benefits of (multi-cycle) schedules



- Effect of low patiences for trees and NNs (depending on the metric)



Importance of preprocessing

Preprocessing is important – standardization is not enough! (Tschalzev et al., 2024)

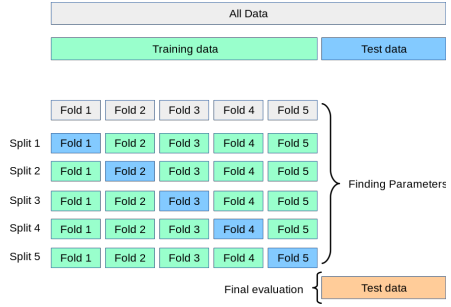
- RealMLP: robust quantile-based rescaling + soft clipping (soon in `skrub`!)
- other good alternative: modified quantile transform as in TabM (Gorishniy et al., 2024) etc.
- With the rise of NNs for tabular data, preprocessing is becoming more important for AutoML (?)

TabArena: A living benchmark for tabular ML

Upcoming paper with Nick Erickson, Lennart Purucker, Andrej Tschalzev, me, Prateek Desai, David Salinas, Frank Hutter

- Online leaderboard: `tabarena.ai`
- Strict selection for tabular predictive IID datasets (51 datasets)
- Strong baselines (optimized search spaces for tree-based models)
- Stronger classification metrics (AUC / log-loss)
- Main aggregation: Elo (a rating system for predicting win-rates vs other algorithms)

Inner and outer cross-validation



Source: https://scikit-learn.org/stable/modules/cross_validation.html

In TabArena:

- Inner cross-validation (train-val for early stopping, hyperparameter tuning): 8-fold as in AutoGluon
- Outer cross-validation (trainval-test for evaluation): 10-repeated 3-fold on small datasets, 3-repeated 3-fold on larger datasets
  - `TabArena-Lite`: only one outer fold

Weighted ensembles (Caruana et al., 2004)

Goal: learn weighted ensemble $f(x) = \frac{\sum_i w_i f_i(x)}{\sum_i w_i}$

- Caruana et al. (2004): Learn weights w_i on validation set by simple greedy algorithm

  for iter in range(n_steps): # e.g. 40 steps
  do $w_i \leftarrow w_i + 1$ for the i that most improves the target metric
  take configuration with best validation score

- Relatively cheap. Some overfitting risk. Increases inference cost.

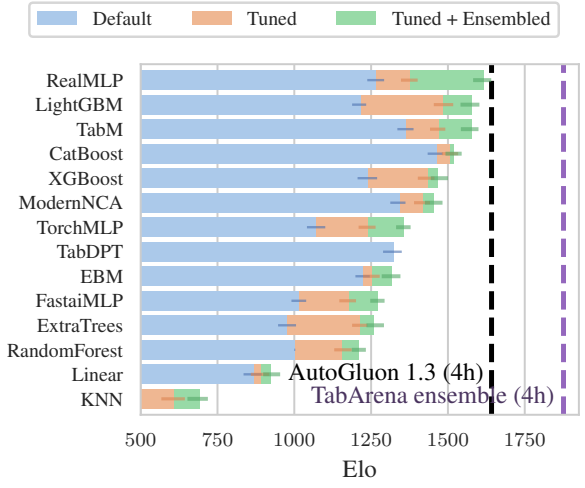
Meta-learned portfolios (e.g., Salinas and Erickson, 2024; Feurer et al., 2022)

- Portfolio: list of (model, hyperparams) that should be trained (in this order)
- Can meta-learn a portfolio:
  - Run *dense* evaluation: same set of (model, hyperparams) for all datasets
  - Greedily pick those (model, hyperparams) that most improve the average portfolio performance

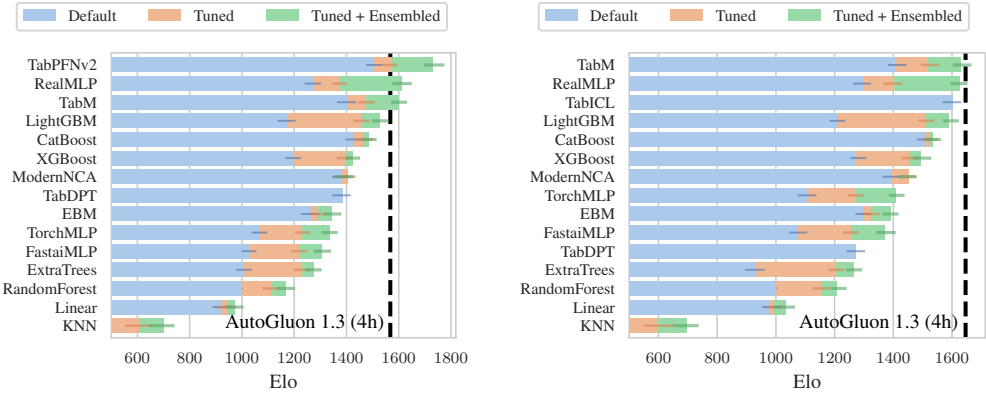
Optimizing RealMLP (and tree baselines) for TabArena

- How to tune the search space itself? With surrogates! *(Publishable? Who finds this interesting?)*
- Resulting changes to the search space
  - Increase/change certain search spaces: numerical embeddings scale, #layers, width
  - Don't tune certain parameters: activations, type of numerical embeddings, whether to use scaling layer
  - Tune new parameters: learning rate factors, β_2
  - Add label smoothing schedule

TabArena: Main results

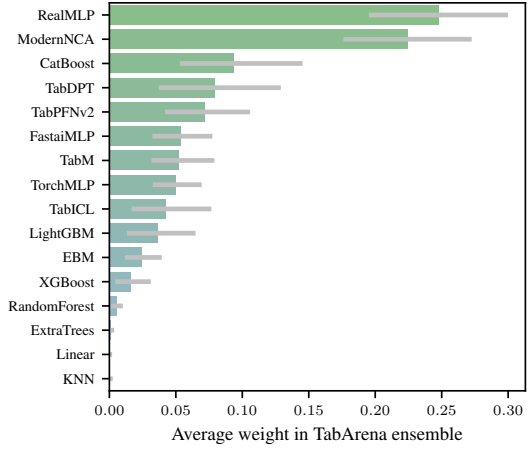


TabArena: Foundation models



Results for TabPFNv2-compatible (left) and TabICL-compatible (right) datasets

TabArena: Average portfolio weights



RealMLP and ModernNCA have large portfolio weights - due to validation set overfitting?

Which differences in TabArena benefit which models?

What benefits RealMLP?

- Better search space
- Inner CV
- Ensembling

What benefits GBDTs?

- Better search spaces
- Many classification datasets + non-accuracy metrics
- No game-like / simulated datasets
- Aggregation (compared to shifted geometric mean)

Classification metrics

- Accuracy-type metrics (Accuracy, F1, balanced accuracy, MCC, . . .): noisy, might need to tune threshold on separate “calibration” set

Classification metrics

- Accuracy-type metrics (Accuracy, F1, balanced accuracy, MCC, . . .): noisy, might need to tune threshold on separate “calibration” set
- AUROC: Better (for binary classification), still requires tuning the threshold afterwards

Classification metrics

- Accuracy-type metrics (Accuracy, F1, balanced accuracy, MCC, ...): noisy, might need to tune threshold on separate "calibration" set
- AUROC: Better (for binary classification), still requires tuning the threshold afterwards
- Proper losses: $\mathbb{E}_{y \sim P} \ell(\hat{P}, y)$ minimized by the true distribution $\hat{P} = P$

Classification metrics

- Accuracy-type metrics (Accuracy, F1, balanced accuracy, MCC, ...): noisy, might need to tune threshold on separate "calibration" set
- AUROC: Better (for binary classification), still requires tuning the threshold afterwards
- Proper losses: $\mathbb{E}_{y\sim P}\ell(\hat{P}, y)$ minimized by the true distribution $\hat{P} = P$
  - Log-loss = cross-entropy = neg-log-likelihood

Classification metrics

- Accuracy-type metrics (Accuracy, F1, balanced accuracy, MCC, ...): noisy, might need to tune threshold on separate "calibration" set
- AUROC: Better (for binary classification), still requires tuning the threshold afterwards
- Proper losses: $\mathbb{E}_{y\sim P}\ell(\hat{P}, y)$ minimized by the true distribution $\hat{P} = P$
  - Log-loss = cross-entropy = neg-log-likelihood
  - Brier loss = MSE = $\|\hat{P} - e_y\|^2$

Classification metrics

- Accuracy-type metrics (Accuracy, F1, balanced accuracy, MCC, ...): noisy, might need to tune threshold on separate "calibration" set
- AUROC: Better (for binary classification), still requires tuning the threshold afterwards
- Proper losses: $\mathbb{E}_{y\sim P}\ell(\hat{P}, y)$ minimized by the true distribution $\hat{P} = P$
  - Log-loss = cross-entropy = neg-log-likelihood
  - Brier loss = MSE = $\|\hat{P} - e_y\|^2$

  Proper losses can often be improved by post-hoc calibration on a validation/calibration set, learning a transformation $g(\hat{P}(x))$

Classification metrics

- Accuracy-type metrics (Accuracy, F1, balanced accuracy, MCC, ...): noisy, might need to tune threshold on separate "calibration" set
- AUROC: Better (for binary classification), still requires tuning the threshold afterwards
- Proper losses: $\mathbb{E}_{y\sim P}\ell(\hat{P}, y)$ minimized by the true distribution $\hat{P} = P$
  - Log-loss = cross-entropy = neg-log-likelihood
  - Brier loss = MSE = $\|\hat{P} - e_y\|^2$

  Proper losses can often be improved by post-hoc calibration on a validation/calibration set, learning a transformation $g(\hat{P}(x))$
  - Potentially some open AutoML questions re what is the best way to apply this with cross-validation etc.

Classification metrics

- Accuracy-type metrics (Accuracy, F1, balanced accuracy, MCC, ...): noisy, might need to tune threshold on separate "calibration" set
- AUROC: Better (for binary classification), still requires tuning the threshold afterwards
- Proper losses: $\mathbb{E}_{y\sim P}\ell(\hat{P}, y)$ minimized by the true distribution $\hat{P} = P$
  - Log-loss = cross-entropy = neg-log-likelihood
  - Brier loss = MSE = $\|\hat{P} - e_y\|^2$

  Proper losses can often be improved by post-hoc calibration on a validation/calibration set, learning a transformation $g(\hat{P}(x))$
  - Potentially some open AutoML questions re what is the best way to apply this with cross-validation etc.

  $\rightarrow$ Perform early stopping + tuning on the post-hoc calibrated loss (Ranjan et al., 2024; Berta et al., 2025)

Outlook

- Done right, MLPs can match / outperform boosted trees, even on smaller datasets!
- More opportunities in classical DL methods / MLPs *(anyone interested?)*, overshadowed by foundation models
- TabICL (Qu et al., 2025): foundation model scaling to larger datasets
- Distributional regression is underexplored?
- AutoML topics: preprocessing / feature engineering, weighted ensembles, stacking, meta-learning, portfolios, ...

`github.com/dholzmueller/pytabkit`
`github.com/soda-inria/tabicl`
`github.com/dholzmueller/probmetrics`

Literature I

Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? *Neural Information Processing Systems*, 2022.

Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045):319–326, 2025. URL https://www.nature.com/articles/s41586-024-08328-6. Publisher: Nature Publishing Group UK London.

David Holzmüller, Leo Grinsztajn, and Ingo Steinwart. Better by default: Strong pre-tuned MLPs and boosted trees on tabular data. In *Neural Information Processing Systems*, 2024.

Andrej Tschalzev, Sascha Marton, Stefan Lüdtke, Christian Bartelt, and Heiner Stuckenschmidt. A data-centric perspective on evaluating machine learning models for tabular data, December 2024. URL http://arxiv.org/abs/2407.02112. arXiv:2407.02112 [cs].

Yury Gorishniy, Akim Kotelnikov, and Artem Babenko. TabM: Advancing Tabular Deep Learning with Parameter-Efficient Ensembling, November 2024. URL http://arxiv.org/abs/2410.24210. arXiv:2410.24210 [cs].

Rich Caruana, Alexandru Niculescu-Mizil, Geoff Crew, and Alex Ksikes. Ensemble selection from libraries of models. In *International Conference on Machine Learning*, 2004.

Literature II

David Salinas and Nick Erickson. TabRepo: A large scale repository of tabular model evaluations and its AutoML applications. In *AutoML Conference*, 2024.

Matthias Feurer, Katharina Eggensperger, Stefan Falkner, Marius Lindauer, and Frank Hutter. Auto-sklearn 2.0: Hands-free automl via meta-learning. *The Journal of Machine Learning Research*, 23(261), 2022.

Rishabh Ranjan, Saurabh Garg, Mrigank Raman, Carlos Guestrin, and Zachary Lipton. Post-hoc reversal: Are we selecting models prematurely?, October 2024. URL http://arxiv.org/abs/2404.07815. arXiv:2404.07815 [cs].

Eugene Berta, David Holzmüller, Michael I. Jordan, and Francis Bach. Rethinking early stopping: Refine, then calibrate. *arXiv preprint arXiv:2501.19195*, 2025. URL https://arxiv.org/abs/2501.19195.

Jingang Qu, David Holzmüller, Gaël Varoquaux, and Marine Le Morvan. TabICL: A Tabular Foundation Model for In-Context Learning on Large Data, February 2025. URL http://arxiv.org/abs/2502.05564. arXiv:2502.05564 [cs].

How to get tabular data?

From OpenML (convenient python API) or the UCI repository (older)

```
import openml

task = openml.tasks.get_task(361267)  # brazilian_housing
X, y, _, _ = task.get_dataset().get_data(
    dataset_format='dataframe',
    target=task.target_name
)

print(X.head(2))
#          city   area  rooms  ...      furniture     hoa
# 0   Sao Paulo   70.0      2  ...      furnished  2065.0
# 1   Sao Paulo  320.0      4  ... not furnished  1200.0
```

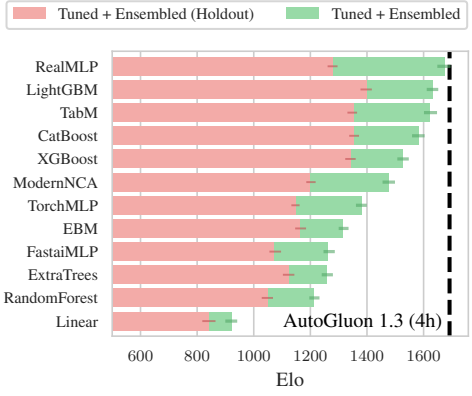
Insights

- Good benchmarks and good baselines are essential for development
- Don't make the benchmark too annoying: Limit #samples, #features, #classes, larger but fewer outer folds
- Use speed optimizations for fast iterations
- Theory can give you ideas but many of them won't work
- Trying optimizers, initializations, and parametrizations is more frustrating than architecture / regularization
- Automate your meta-learning process, it's easier to justify and less of a headache
- Reviewers like "fair" ablations and transferable components (need to demonstrate!)
- Reviewers don't like bag-of-tricks (but they should!)
- Reviewers don't like having multiple benchmarks with different baselines (even though they should like it more than a single benchmark)

Fundamental dilemmas/questions in benchmarking

- Development benchmark (fast) vs. evaluation benchmark
- Architectures / fair ablations vs. systems
- Time measurement vs. parallelization
- Results imputation
- Measuring models at their best vs. at their worst (classification metrics, etc.)
- How to manage random seeds
- Type of CV
- Type of HPO $\rightarrow$ different downstream opportunities

Bias introduced by holdout validation



TabArena: runtimes

